

TFG — Engineering Requirements Specification

1. Purpose

In this TFG I will build a web application that demonstrates **Fixation Disparity Curves (FDC)** and automatically **classifies** a patient's curve into **Type 1–Type 4** using **polynomial approximation** and **error comparison**.

All curve fitting and error computation will be implemented in the **backend**, and the frontend will consume the backend via **REST APIs** to display the results.

The curve models and error calculation must follow the method described in:
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12682111/pdf/OPO-45-1642.pdf>

2. Scope

Phase 1 (in scope)

I will implement:

- A **backend service** that:
 - receives a dataset of 7 patient responses,
 - fits the data to **four candidate curve models** (Type 1–4),
 - computes an **error value** for each type according to the reference paper,
 - returns the fitted curves, errors, and final classification.
- A **frontend web app** that:
 - collects input (7 y-values),
 - calls the backend API,
 - renders:
 - the original measured points,
 - all four fitted curves in one chart,
 - errors for each type,
 - and the predicted curve type.

Phase 2 (out of scope for now)

To be defined after validation of Phase 1.

3. Data Model & Constraints

3.1 Dataset definition

- The **x-values are fixed** clinical stimulus values and are the same for all patients.
- The user provides **7 y-values** (patient responses).
- Each dataset is therefore:
 - $x = [x1 \dots x7]$ (predefined constant)
 - $y = [y1 \dots y7]$ (provided by the user)

3.2 Curve types

The system must evaluate four candidate curve types:

- Type 1
- Type 2
- Type 3
- Type 4

Each type corresponds to a polynomial-based approximation model defined in the reference documentation.

3.3 Classification rule

- Fit all four curve types to the dataset
- Compute errors $E1 \dots E4$
- Predicted type = $\text{argmin}(E1..E4)$

Tie case handling must be deterministic (see FR-6).

4. System Architecture (Phase 1)

4.1 Backend (calculation layer)

Responsibilities:

- store the fixed x-values (constants)
- perform model fitting for each curve type
- compute errors based on the paper

- return structured results to the frontend

Principle: All math + model logic stays in the backend to ensure correctness, maintainability, and consistent results.

4.2 Frontend (presentation layer)

Responsibilities:

- collect user inputs
 - validate input format (basic numeric validation)
 - call backend APIs
 - render chart + results
 - apply UPC-inspired UI theme
-

5. Functional Requirements

FR-1 — Input of Patient Responses (Frontend)

Description: The frontend must allow the user to input 7 numeric y-values corresponding to the fixed x-values.

Acceptance Criteria:

- User can enter exactly 7 values.
 - Non-numeric values are rejected with a clear message.
 - UI shows the fixed x-values next to each y input to avoid confusion.
-

FR-2 — Compute Endpoint (Backend)

Description: The backend must expose an endpoint that accepts 7 y-values and returns the computed model results.

Acceptance Criteria:

- Endpoint accepts a JSON request containing:
 - `y: number[7]`
 - optional `datasetId` or `notes` (optional)
- Backend validates:
 - `array length == 7`
 - all values are finite numbers

- Returns results for all types and the classification result (see FR-3, FR-4).
-

FR-3 — Model Fitting for Each Type (Backend)

Description: For each curve type (1–4), the backend fits a polynomial-based model to the provided points.

Acceptance Criteria:

- Backend outputs, for each type:
 - model coefficients/parameters
 - predicted y-values for the 7 x-values
 - optional dense sampled points for smooth plotting (recommended)

If a fit fails for a type:

- that type returns an error entry with a readable reason
 - overall request still succeeds if at least one type succeeds
-

FR-4 — Error Computation (Backend)

Description: Backend computes an error value for each curve type using the method defined in the reference paper.

Acceptance Criteria:

- Backend returns four error values (or `null` if not computable):
 - `error.type1`, `error.type2`, `error.type3`, `error.type4`
 - Errors must be comparable (same metric / scale).
 - Error calculation must match the referenced method.
-

FR-5 — Classification Output (Backend)

Description: Backend selects the best curve type based on minimum error.

Acceptance Criteria:

- Backend returns:
 - `predictedType: 1|2|3|4`
 - `minError: number`

- In tie cases, backend applies a deterministic tie-break rule:
 - Example: choose smallest type number OR return `predictedType: null` and `ties: [...]`
 - Tie strategy must be documented and consistent.
-

FR-6 — Visualization (Frontend)

Description: The frontend must render a chart containing:

- the 7 original data points
- the 4 fitted curves (Type 1–4), each in a distinct color

Acceptance Criteria:

- One graph showing:
 - 7 measured points (scatter)
 - 4 fitted curves (line)
 - Legend maps color → curve type
 - Axis labels:
 - x: disparity stimulus
 - y: patient response
 - Graph updates after successful API response.
-

FR-7 — Results Panel (Frontend)

Description: Display error values and the predicted type below the chart.

Acceptance Criteria:

- UI shows:
 - Error for Type 1–4
 - Predicted type
 - The minimum error row is visually highlighted.
 - If a type fails to compute, UI shows `N/A` and the failure reason.
-

FR-8 — Example Datasets (Frontend + Backend)

Description: Provide example datasets for demonstration and testing.

Acceptance Criteria:

- The app includes at least 3 example datasets selectable from the UI.
- Selecting one auto-fills y-values and triggers computation.

Implementation options:

- frontend hard-coded examples, or
 - backend endpoint `/examples` returning sample datasets.
-

6. API Specification (Phase 1)

6.1 POST `/api/v1/compute`

Request

```
{  
  "y": [0.1, 0.2, 0.0, -0.1, -0.2, -0.1, 0.0]  
}
```

Response (example structure)

```
{  
  "x": [-3, -2, -1, 0, 1, 2, 3],  
  "measured": {  
    "y": [0.1, 0.2, 0.0, -0.1, -0.2, -0.1, 0.0]  
  },  
  "models": {  
    "type1": {  
      "coefficients": [/* ... */],  
      "predictedY": [/* 7 values */],  
    }  
  }  
}
```

```
    "curve": [{ "x": -3, "y": 0.12}, { "x": -2.9, "y": 0.11} ]
  },
  "type2": { "..." : "..." },
  "type3": { "..." : "..." },
  "type4": { "..." : "..." }
},
"errors": {
  "type1": 0.123,
  "type2": 0.085,
"type3": 0.140,
  "type4": 0.102
},
"classification": {
  "predictedType": 2,
  "minError": 0.085,
  "ties": []
}
}
```

6.2 GET `/api/v1/examples` (recommended)

Returns predefined example datasets.

6.3 GET `/api/v1/health`

Returns service status for monitoring/dev sanity checks.

7. Non-Functional Requirements

NFR-1 — Performance

- API compute must finish quickly for interactive usage (target: < 200ms for normal input).
- Frontend should update UI instantly after response.

NFR-2 — Robustness

- Backend must validate input and provide clear error messages.
- App must not crash on invalid input.

NFR-3 — Maintainability & Structure

- Clear separation of modules:
 - curve type definitions
 - fitting algorithm
 - error computation
 - API handlers
 - shared constants (fixed x-values)

NFR-4 — Determinism

- The same input must always produce the same output.

NFR-5 — Reproducibility

- Reference paper and formula decisions are documented in the repository.
-

8. UI / Design Requirements (UPC Theme)

The frontend should follow a **UPC-inspired** design:

- clean academic look
- consistent color palette and typography
- layout optimized for clarity (input → graph → results)

Exact theme can be refined, but Phase 1 must implement a consistent baseline.

9. Testing Requirements

Backend Unit Tests

- Fit functions: correct output shape + finite values
- Error computation: deterministic output for known examples
- Classification: correct minimum selection + tie behavior

Backend Integration Tests

- `POST /compute` returns expected schema
- invalid inputs return `400` with meaningful errors

Frontend Tests (minimum)

- input validation
 - rendering with mocked API response
-

10. Deliverables (Phase 1)

- Backend service:
 - polynomial fitting for Types 1–4
 - error computation per type
 - classification result
 - documented REST API
 - Frontend app:
 - input UI
 - chart visualization
 - errors + predicted type display
 - UPC-inspired theme baseline
 - Documentation:
 - setup instructions
 - API docs
 - explanation of curve-type selection logic
 - Tests for core backend logic
-

11. Milestones (suggested)

1. Repo scaffold (frontend + backend) + CI basic
2. Fixed x-values + input UI
3. Backend compute endpoint returning mock results
4. Chart rendering in frontend (with mocked data)
5. Implement real fitting (Type 1–4)
6. Implement errors + classification
7. Example datasets + polish
8. Tests + final documentation